TP2 — Mini-projet IA : étude de cas appliquée

Matière : Les fondamentaux de l'IA **Niveau :** Bachelor 3 **Durée :** 3h **Objectifs pédagogiques :** - Cadrer un problème métier en objectif ML - Construire un pipeline complet sur données réelles (CSV/JSON) - Évaluer et argumenter les choix de modèles - Intégrer la conformité AI Act dans la démarche

Contexte

Ce TP est un **mini-projet autonome**. Vous choisissez un cas parmi trois, construisez un pipeline ML complet sur des données réelles et produisez une analyse incluant la conformité réglementaire. Chaque étape est guidée mais laisse place à l'initiative.

Prérequis

```bash
pip install scikit-learn matplotlib pandas numpy seaborn shap xgboost requests
```

Ou directement sur **Google Colab** (aucune installation) : colab.research.google.com

Cas proposés — choisissez-en un

Cas A — Churn clients (tabulaire, classification binaire)

Objectif métier : Prédire si un client va résilier son abonnement télécom. **Dataset :** IBM Telco Customer Churn — 7 043 clients, 20 variables (durée contrat, facture mensuelle, services souscrits...) **URL :** `https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv`

Cas B — Avis produits (NLP, analyse de sentiments)

Objectif métier : Classifier automatiquement des avis clients Amazon en positif (1) ou négatif (0). **Dataset :** UCI Sentiment Labelled Sentences — 1 000 avis Amazon annotés manuellement. **URL :** `https://archive.ics.uci.edu/ml/machine-learning-databases/00331/sentiment%20labelled%20sentences.zip`

Cas C — Cybersécurité (logs réseau, détection d'intrusions)

Objectif métier : Détecter des connexions malveillantes dans des logs réseau (attaque vs trafic normal). **Dataset :** NSL-KDD — référence académique pour la détection d'intrusions réseau. **URL :** `https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain+_20Percent.txt`

Étape 1 — Cadrage du projet *(15 min)*

Avant de coder, définissez précisément le problème. Remplissez ce tableau pour votre cas :

DIMENSION	VOTRE RÉPONSE
Problème métier	Quelle décision business est visée ?
KPI métier	Comment mesurer le succès côté business ?
Variable cible (y)	Quelle colonne prédire ? Quelles valeurs possibles ?
Type de tâche ML	Classification binaire / multi-classe / régression ?
Métrique ML principale	Accuracy ? F1-macro ? Recall ? Justifiez ce choix.
Risques identifiés	Biais potentiels ? Variables sensibles ?
Niveau AI Act	**Inacceptable / Haut risque / Risque limité / Minimal ?**

Questions guidées : - Pour votre cas, une erreur de type **faux négatif** ou **faux positif** est-elle plus coûteuse ? Pourquoi ? - Quelles populations pourraient être discriminées si le modèle est biaisé ?

Étape 2 — Préparation des données *(~30 min)*

À la fin de cette étape, vous devez avoir défini : `X_train_sc`, `X_test_sc`, `y_train`, `y_test`, `feature_names`. Les étapes 3 à 6 utilisent ces variables de façon uniforme.

Cas A — Churn clients

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Chargement depuis URL publique — IBM Telco Customer Churn (données réelles)
url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
df  = pd.read_csv(url)

print(f"Forme : {df.shape}")
print(f"\nDistribution cible :")
print(df['Churn'].value_counts())
print(f"\nColonnes : {list(df.columns)}")

# Nettoyage : TotalCharges contient des espaces (clients récents sans charges)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df.dropna(inplace=True)
print(f"\nAprès nettoyage : {df.shape}")

# Encodage : variables catégorielles en dummies
df_enc = pd.get_dummies(df.drop('customerID', axis=1), drop_first=True)
X = df_enc.drop('Churn_Yes', axis=1)
y = df_enc['Churn_Yes'].astype(int)
feature_names = list(X.columns)

# Déséquilibre de classes
print(f"\nTaux de churn : {y.mean()*100:.1f}% → dataset déséquilibré")
print("→ Utiliser F1-macro plutôt qu'accuracy seule")

# Split stratifié pour respecter la proportion de churn
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Normalisation
scaler     = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

print(f"\nTrain : {len(X_train)} | Test : {len(X_test)}")
print(f"Nombre de features : {len(feature_names)}")
```

Cas B — Avis produits (NLP)

```python
import requests, zipfile, io
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Chargement depuis UCI ML Repository (archive officielle)
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/00331/sentiment%20labelled%20sentences.zip"
r   = requests.get(url, timeout=30)
z   = zipfile.ZipFile(io.BytesIO(r.content))

# Extraction des avis Amazon (également disponibles : yelp, imdb)
with z.open('sentiment labelled sentences/amazon_cells_labelled.txt') as f:
    df = pd.read_csv(f, sep='\t', header=None, names=['avis', 'sentiment'])

print(f"Forme : {df.shape}")
print(f"Distribution : {df['sentiment'].value_counts().to_dict()}")
print(f"\nExemples :")
print(df.sample(3, random_state=42))

# Feature engineering NLP : TF-IDF (unigrammes + bigrammes)
vectorizer = TfidfVectorizer(max_features=500, stop_words='english', ngram_range=(1, 2))
X = vectorizer.fit_transform(df['avis'])  # Sparse matrix
y = df['sentiment']
feature_names = list(vectorizer.get_feature_names_out())

print(f"\nVocabulaire TF-IDF : {len(feature_names)} tokens")

# Split stratifié
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# TF-IDF est déjà normalisé — pas de StandardScaler
X_train_sc = X_train
X_test_sc  = X_test

print(f"Train : {X_train.shape[0]} | Test : {X_test.shape[0]}")
```

Cas C — Cybersécurité (NSL-KDD)

```python
```

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Colonnes du dataset NSL-KDD (41 features + label + difficulté)
columns = [
    'duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes',
    'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins', 'logged_in',
    'num_compromised', 'root_shell', 'su_attempted', 'num_root',
    'num_file_creations', 'num_shells', 'num_access_files', 'num_outbound_cmds',
    'is_host_login', 'is_guest_login', 'count', 'srv_count', 'serror_rate',
    'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate', 'same_srv_rate',
    'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
    'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
    'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
    'dst_host_serror_rate', 'dst_host_srv_serror_rate',
    'dst_host_rerror_rate', 'dst_host_srv_rerror_rate', 'label', 'difficulty'
]

# Chargement depuis GitHub (dépôt public NSL-KDD, sous-ensemble 20%)
url = "https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain+_20Percent.txt"
df  = pd.read_csv(url, header=None, names=columns)

# Simplification : tâche binaire — normal (0) vs attaque (1)
df['label_binary'] = (df['label'] != 'normal').astype(int)
print(f"Forme : {df.shape}")
print(f"Normal : {(df['label_binary']==0).sum()} | Attaque : {(df['label_binary']==1).sum()}")
print(f"\nTypes d'attaques :")
print(df[df['label'] != 'normal']['label'].value_counts().head(10))

# Encodage des variables catégorielles (protocol, service, flag)
for col in ['protocol_type', 'service', 'flag']:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])

X = df.drop(['label', 'difficulty', 'label_binary'], axis=1)
y = df['label_binary']
feature_names = list(X.columns)

# Split stratifié
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Normalisation
scaler     = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

print(f"\nTrain : {len(X_train)} | Test : {len(X_test)}")
print(f"Nombre de features : {len(feature_names)}")
```

Questions (tous les cas — après Étape 2) : 1. Votre dataset est-il équilibré ? Quelle proportion de chaque classe ? 2. Y a-t-il des variables potentiellement sensibles (âge, localisation, genre…) ? Faut-il les exclure ? 3. Quel prétraitement avez-vous appliqué et pourquoi (encodage, normalisation, TF-IDF) ?

Étape 3 — Modélisation : 3 modèles à comparer *(~45 min)*

PYTHON

```python
import matplotlib.pyplot as plt   # Nécessaire pour tous les cas (A, B et C)
import seaborn as sns              # Nécessaire pour tous les cas (A, B et C)
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score

# Les trois modèles imposés par le TP
modeles = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Random Forest":       RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost":             XGBClassifier(
                               n_estimators=100, random_state=42,
                               eval_metric='logloss', verbosity=0
                           ),
}

resultats = {}

for nom, modele in modeles.items():
    modele.fit(X_train_sc, y_train)
    pred   = modele.predict(X_test_sc)
    acc    = accuracy_score(y_test, pred)
    f1_mac = f1_score(y_test, pred, average='macro')
    f1_wei = f1_score(y_test, pred, average='weighted')
    resultats[nom] = {'accuracy': acc, 'f1_macro': f1_mac, 'f1_weighted': f1_wei}
    print(f"{nom:22s} → Accuracy : {acc*100:.1f}%  |  F1-macro : {f1_mac:.3f}  |  F1-weighted :
{f1_wei:.3f}")

# Tableau comparatif
df_resultats = pd.DataFrame(resultats).T
print("\n=== TABLEAU COMPARATIF ===")
print(df_resultats.round(3))

# Visualisation
df_resultats[['accuracy', 'f1_macro']].plot(kind='bar', figsize=(9, 4))
plt.title('Comparaison des modèles')
plt.ylabel('Score')
plt.xticks(rotation=20)
plt.ylim(0.5, 1.0)
plt.grid(axis='y', alpha=0.5)
plt.tight_layout()
plt.savefig('comparaison_modeles.png')
plt.show()
```

Questions : 1. Quel modèle obtient les meilleures performances sur votre cas ? 2. La Régression Logistique est-elle compétitive malgré sa simplicité ? 3. Quel modèle choisiriez-vous pour un

déploiement en production ? Justifiez votre choix en tenant compte de : performance, interprétabilité, temps d'inférence, coût de maintenance.

Étape 4 — Évaluation approfondie *(~30 min)*

PYTHON

```python
from sklearn.metrics import confusion_matrix, classification_report

# Choisir le meilleur modèle selon votre tableau comparatif
# Remplacez "XGBoost" par le nom du modèle que vous avez identifié
NOM_MEILLEUR = "XGBoost"
meilleur = modeles[NOM_MEILLEUR]
y_pred   = meilleur.predict(X_test_sc)

# Rapport complet
print(f"=== RAPPORT DÉTAILLÉ — {NOM_MEILLEUR} ===")
print(classification_report(y_test, y_pred))

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title(f'Matrice de confusion — {NOM_MEILLEUR}')
plt.ylabel('Réalité')
plt.xlabel('Prédiction')
plt.tight_layout()
plt.savefig('confusion_matrix.png')
plt.show()

# Évolution de Random Forest selon le nombre d'arbres
n_estimators_range = [10, 25, 50, 100, 200]
scores_rf = []

for n in n_estimators_range:
    rf_temp = RandomForestClassifier(n_estimators=n, random_state=42)
    rf_temp.fit(X_train_sc, y_train)
    pred_temp = rf_temp.predict(X_test_sc)
    scores_rf.append(f1_score(y_test, pred_temp, average='macro'))

plt.figure(figsize=(8, 4))
plt.plot(n_estimators_range, scores_rf, 'g-o')
plt.xlabel("Nombre d'arbres (n_estimators)")
plt.ylabel("F1-score macro")
plt.title("Évolution des performances — Random Forest")
plt.grid(True)
plt.tight_layout()
plt.savefig('rf_evolution.png')
plt.show()
```

Analyse critique obligatoire : 1. Identifiez les **faux positifs** et **faux négatifs** dans votre matrice de confusion. 2. Dans votre contexte métier (churn / sentiment / intrusion), lequel est le plus coûteux ? Pourquoi ? 3. Votre meilleur modèle est-il suffisant pour un déploiement réel ? Quelles garanties supplémentaires seraient nécessaires ?

Étape 5 — Explicabilité SHAP *(~20 min, si temps disponible)*

```python
import shap
import scipy.sparse as sp

# SHAP avec le Random Forest (TreeExplainer, rapide sur forêts)
rf_model = modeles["Random Forest"]

# Convertir en dense si sparse (Cas B — TF-IDF)
# Limiter à 200 exemples pour la rapidité
X_sample = X_test_sc[:200]
if sp.issparse(X_sample):
    X_sample = X_sample.toarray()
else:
    X_sample = np.array(X_sample)

explainer   = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_sample)

# Compatibilité SHAP < 0.42 (liste) et SHAP >= 0.42 (tableau 3D ou 2D)
# Classification binaire : on cible la classe positive
if isinstance(shap_values, list):
    sv = shap_values[1]           # SHAP < 0.42 : liste [classe_0, classe_1]
elif len(shap_values.shape) == 3:
    sv = shap_values[:, :, 1]     # SHAP >= 0.42 : tableau (n, features, classes)
else:
    sv = shap_values              # Déjà 2D (cas binaire simplifié)

# Summary plot : top 15 variables les plus influentes
plt.figure()
shap.summary_plot(
    sv, X_sample,
    feature_names=feature_names,
    max_display=15,          # Limiter l'affichage (utile pour NLP)
    show=False
)
plt.title("SHAP — Top 15 variables les plus influentes (classe positive)")
plt.tight_layout()
plt.savefig('shap_summary.png', bbox_inches='tight')
plt.show()

# Cas B (NLP) : les features SHAP = des mots => très interprétable !
# Les mots à droite (valeurs SHAP positives) prédisent les avis positifs
# Les mots à gauche (valeurs SHAP négatives) prédisent les avis négatifs
```

Questions : 1. Quelles sont les **3 variables les plus influentes** selon SHAP ? 2. Ces variables sont-elles cohérentes avec votre intuition métier ? *(Cas B : les mots identifiés correspondent-ils à des indicateurs évidents de satisfaction ?)* 3. Si une variable sensible (âge, genre, origine…) apparaît parmi les plus importantes, que faut-il faire ?

Étape 6 — Conformité AI Act *(~20 min)*

Remplissez ce tableau de conformité pour votre cas :

CRITÈRE	VOTRE ANALYSE
Niveau de risque AI Act	Inacceptable / Haut risque / Risque limité / Minimal
Justification du niveau	Pourquoi ? (secteur, impact sur personnes, décision automatisée ?)
Base légale RGPD	Consentement / Contrat / Intérêt légitime ?
DPIA requis ?	Analyse d'impact sur la vie privée nécessaire ? Pourquoi ?
Explicabilité requise	Le modèle doit-il justifier ses décisions ? Comment ?
Supervision humaine	Un humain doit-il valider chaque décision ?
Audit et traçabilité	MLflow, versioning des modèles, logs de décisions ?
Droits des personnes	**Accès, rectification, opposition aux décisions automatisées ?**

Questions : 1. Votre système nécessite-t-il un audit de conformité **avant** déploiement ? Par qui ? 2. Les personnes affectées par les décisions ont-elles un **droit de recours** ? 3. Quel organisme de contrôle surveille ce type de système dans votre secteur (CNIL, ANC, ANSSI...) ?

Livrables

LIVRABLE	DESCRIPTION
Notebook propre/exécutable	Toutes les étapes exécutées, cellules commentées, sans erreurs
Graphiques clés	Matrice de confusion + comparatif modèles + SHAP summary
Slide synthèse	**1 slide au format ci-dessous**

Template slide synthèse

```
┌──────────────────────────────────────────────────────────────────┐
│  CAS : [A / B / C]           MODÈLE RETENU : [...]               │
├──────────────────────────────────────────────────────────────────┤
│  MÉTRIQUES FINALES                                               │
│  Accuracy : ...%   F1-macro : ...   F1-weighted : ...            │
├──────────────────────────────────────────────────────────────────┤
│  TOP 3 VARIABLES (SHAP)                                          │
│  1. [variable]   2. [variable]   3. [variable]                   │
├──────────────────────────────────────────────────────────────────┤
│  AI ACT : Niveau [...]  —  [Justification en 1 phrase]           │
├──────────────────────────────────────────────────────────────────┤
│  LIMITE PRINCIPALE IDENTIFIÉE : [...]                            │
└──────────────────────────────────────────────────────────────────┘
```

Critères d'évaluation

CRITÈRE	POINTS
Cadrage métier : tableau complet et pertinent	3
Préparation données : nettoyage, gestion du déséquilibre	3
Modélisation : comparaison 3 modèles argumentée	4
Évaluation : analyse confusion matrix + métriques	4
Explicabilité SHAP (capture + analyse)	3
Conformité AI Act : tableau + argumentation	3
Total	**20**

Ressources

- scikit-learn — Documentation
- XGBoost — Documentation
- SHAP — Documentation
- AI Act — Texte officiel EUR-Lex
- CNIL — RGPD et IA
- Dataset Telco Churn : IBM telco-customer-churn-on-icp4d

- Dataset Avis Amazon : UCI Sentiment Labelled Sentences
- Dataset NSL-KDD : defcom17/NSL_KDD sur GitHub